

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

Microservicio TiendaTech

GA-SUM-03: PE-U2

Arquitectura, API y Contenerización del Sistema

Asignatura: Aplicaciones Distribuidas

Curso: 7mo Software – A

Profesor: Ing. Guerrero Ulloa Gleiston Cicerón

Fecha: 17 de junio de 2026

Integrantes:

Jhinson Stalyn Aucatoma Celorio

Jeremy Alexis Alvarez Parraga

Carlos Jose Carpio Mendoza

Repositorio: https://github.com/JhinsonAucatoma/-GA_SUM_03

Abstract

This work describes the design, implementation, and deployment of a microservices architecture for the distributed application **TiendaTech**. The solution is composed of three independent microservices responsible for user authentication, resource management, and notification logging. To ensure secure operations, JSON Web Token (JWT) authentication was implemented, while an Nginx-based API Gateway centralizes access to the services. In addition, each microservice has its own database, promoting loose coupling and service independence. Deployment was carried out using Docker and Docker Compose, enabling all services to run in a coordinated containerized environment. Finally, functional tests were performed to verify the correct behavior of the implemented processes. The obtained results demonstrate that the developed architecture is a suitable alternative for building scalable, organized, and maintainable distributed applications.

Resumen

El presente trabajo describe el diseño, implementación y despliegue de una arquitectura de microservicios para la aplicación distribuida **TiendaTech**. La solución se encuentra conformada por tres microservicios independientes encargados de la autenticación de usuarios, la gestión de recursos y el registro de notificaciones. Para garantizar la seguridad de las operaciones se implementó autenticación mediante JSON Web Tokens (JWT), mientras que un API Gateway basado en Nginx centraliza el acceso a los servicios. Asimismo, cada microservicio dispone de una base de datos propia, favoreciendo el desacoplamiento y la independencia de los componentes. El despliegue se realizó mediante Docker y Docker Compose, permitiendo ejecutar todos los servicios de manera coordinada dentro de contenedores. Finalmente, se llevaron a cabo pruebas funcionales para verificar el correcto comportamiento de los diferentes procesos implementados. Los resultados obtenidos demuestran que la arquitectura desarrollada constituye una alternativa adecuada para la construcción de aplicaciones distribuidas escalables y de fácil mantenimiento.

1. Introducción

Las arquitecturas distribuidas han adquirido una gran importancia en el desarrollo de software moderno debido a la necesidad de construir sistemas capaces de soportar un mayor número de usuarios, facilitar el mantenimiento y adaptarse a los cambios de manera eficiente. Dentro de este contexto, la arquitectura de microservicios se ha consolidado como una alternativa ampliamente utilizada, ya que permite dividir una aplicación en servicios independientes que pueden desarrollarse, desplegarse y escalarse de forma individual.

El presente trabajo corresponde a la implementación de **TiendaTech**, una aplicación basada en microservicios que integra mecanismos de autenticación, gestión de recursos y registro de notificaciones. La solución incorpora un API Gateway para centralizar el acceso a los servicios, autenticación mediante JWT para proteger los recursos y una estrategia de contenerización basada en Docker y Docker Compose para simplificar el despliegue de la aplicación.

A lo largo del desarrollo se aplicaron conceptos relacionados con arquitecturas distribuidas, APIs REST, seguridad y contenerización, permitiendo construir un entorno funcional que demuestra las ventajas de la separación de responsabilidades y la independencia entre servicios. Además, se realizaron pruebas de funcionamiento para verificar la correcta interacción entre los componentes y validar el cumplimiento de los requisitos establecidos en la práctica.

2. Evolución Arquitectónica: Monolito → SOA → Microservicios

La evolución de las arquitecturas de software ha estado marcada por la necesidad de mejorar la escalabilidad, el mantenimiento y la capacidad de adaptación de las aplicaciones. Este recorrido puede comprenderse como una progresión de tres

paradigmas que se suceden y complementan entre sí: la arquitectura monolítica, la Arquitectura Orientada a Servicios (SOA) y los microservicios.

Inicialmente predominó la **arquitectura monolítica**, en la que todos los componentes del sistema se desarrollaban y desplegaban como una única unidad. Este enfoque facilitaba el desarrollo inicial, pero dificultaba el escalado y las actualizaciones, ya que cualquier cambio requería volver a desplegar toda la aplicación [1].

Para superar estas limitaciones surgió la **Arquitectura Orientada a Servicios (SOA)**, que propuso dividir las aplicaciones en servicios independientes que se comunican mediante mensajes. Aunque este modelo mejoró el desacoplamiento funcional, muchas implementaciones continuaron dependiendo de despliegues centralizados, limitando su flexibilidad operativa [2].

Posteriormente aparecieron los **microservicios**, considerados una evolución de SOA. En este modelo, cada servicio puede desarrollarse, desplegarse y escalarse de forma independiente, permitiendo una mayor flexibilidad, tolerancia a fallos y aprovechamiento de recursos. Además, esta arquitectura favorece la autonomía de los equipos de desarrollo y se adapta mejor a los entornos cloud modernos [1], [2].

En consecuencia, la transición desde arquitecturas monolíticas hacia microservicios representa una evolución tanto técnica como organizacional, orientada a construir sistemas más escalables, resilientes y capaces de responder con rapidez a los cambios del negocio.

Comparación entre arquitecturas

La siguiente tabla resume las diferencias más relevantes entre las tres arquitecturas, considerando tiempos de despliegue, escalabilidad, aislamiento de fallos y costos de infraestructura.

Cuadro 1: *Comparativa entre las arquitecturas: Monolito, SOA y Microservicios.*

Característica	Arquitectura Monolítica	SOA	Microservicios
Tiempos de Despliegue	Lentos y complejos. Se despliega como una unidad inseparable [1]. Requiere ventanas de mantenimiento con tiempos de inactividad [3].	Moderados pero rígidos. A menudo se comportan como “monolitos de despliegue”, donde todos los servicios deben lanzarse a la vez [2].	Rápidos e independientes. Permite despliegue continuo y actualizaciones sin tiempo de inactividad. Las unidades (contenedores) inician en segundos o milisegundos [2].
Escalabilidad	Ineficiente. Requiere escalar toda la aplicación aunque solo un componente tenga carga [1]. Limitada a la capacidad del hardware más potente (escalado vertical) [4].	Limitada por cuellos de botella. El escalado suele depender de un bus centralizado (ESB) que requiere servidores de alto costo [5].	Alta y granular. Escalado horizontal independiente por servicio. En pruebas de alta carga (300k req.), superan al monolito procesando 239 vs 180 req/s [1].
Aislamiento de Fallos	Nulo o Mínimo. Un error en un módulo puede colapsar todo el proceso ejecutable [1].	Bajo. La dependencia de un bus central y gobernanza global predetermina fallos en cadena si el mecanismo de integración falla [6].	Excelente. El acoplamiento suelto permite que los errores permanezcan aislados, permitiendo que otros servicios funcionen sin perturbaciones [1].
Costos e Infraestructura	Altos en la nube. El escalado ineficiente genera facturas crecientes [2].	Elevados. Escalar el ESB requiere hardware de especificaciones muy altas, aumentando considerablemente el gasto [5].	Reducción significativa. Pueden reducir costos de infraestructura en un 13 % comparado con monolitos estándar, y hasta un 77 % si se diseñan para entornos serverless/Lambda [4].

Principios SOA

La Arquitectura Orientada a Servicios se fundamenta en siete principios de diseño que determinan cómo deben construirse e interactuar los servicios para garantizar su interoperabilidad y reutilización a largo plazo.

Cuadro 2: *Principios de la Arquitectura Orientada a Servicios (SOA)* [7].

Principio	Descripción
Reusability (Reutilización)	Los servicios se diseñan para ser utilizados en el futuro cercano o distante mediante el uso de estándares libres de plataforma e interfaces de servicio [7].
Standardized Service Contract (Contrato de servicio estandarizado)	Es el único medio para reconocer a los receptores del servicio y debe incluir dirección, nombre, función y datos específicos de entrada y salida [7].
Loose Coupling (Acoplamiento débil)	Los servicios deben ser lo más independientes posible entre sí, permitiendo que uno utilice información de otro sin depender de su implementación interna [7].
Hiding of Internal Implementation / Encapsulation (Encapsulamiento)	Solo la interfaz es observable por los receptores; los usuarios deben conocer el rendimiento del servicio, pero no su tecnología o implementación interna [7].
Service Composability (Componibilidad)	Capacidad de los servicios para combinarse entre sí y cubrir las funciones o el rendimiento de un sub-sistema completo [7].
Service Autonomy & Statelessness (Autonomía y falta de estado)	Un servicio debe ser autónomo de su contexto de ejecución y no debe guardar su estado actual para poder responder a múltiples solicitudes simultáneas [7].
Discoverability (Descubribilidad)	La interfaz del servicio debe permitir que los usuarios la deriven, descubran y extraigan fácilmente para que el concepto de SOA trascienda los límites organizacionales [7].

Los 9 principios de los microservicios

Los **9 principios de los microservicios** (propuestos originalmente por Martin Fowler) definen las características que distinguen una arquitectura de microservicios bien diseñada. La siguiente tabla los presenta de forma estructurada para facilitar su comparación y estudio.

Cuadro 3: *Los nueve principios de los microservicios según Martin Fowler [8].*

Principio	Descripción
Componentización vía servicios (Componentization via services)	Se refiere a la creación de pequeños microservicios que, por diseño, son desplegados de forma independiente y se exponen como una “caja negra”, sin que el consumidor necesite conocer el lenguaje o la tecnología con la que fueron creados [8].
Organizado en torno a capacidades de negocio (Organized around business capabilities)	La división modular de los servicios debe basarse en las capacidades funcionales del negocio y no en capas técnicas tradicionales como presentación, lógica de negocio o acceso a datos [8].
Gobernanza descentralizada (Decentralized governance)	Los microservicios deben ser autónomos, con equipos de trabajo responsables de su propio grupo de servicios, permitiendo incluso el uso de diferentes estándares tecnológicos según sea conveniente para cada producto [8].
Automatización de la infraestructura (Infrastructure automation)	Implica el uso de herramientas DevOps para automatizar la integración y el despliegue continuo (CI/CD), permitiendo liberar nuevas versiones de forma frecuente, controlada y con pruebas automáticas [8].
Diseño para el fallo (Design for failure)	Los servicios se diseñan para ser tolerantes a fallos; si un microservicio falla, el resto de la aplicación debe continuar funcionando. Se utilizan herramientas de monitoreo para detectar caídas y levantar automáticamente los servicios afectados [8].
Productos, no proyectos (Products not projects)	Se fomenta la creación de equipos multifuncionales (full-stack) que se encargan de todo el ciclo de vida del software, desde la construcción hasta el mantenimiento y soporte en el entorno real [8].
Endpoints inteligentes y tuberías tontas (Smart endpoints and dumb pipes)	Se recomienda que la inteligencia resida en los puntos finales (el servicio transforma datos y aplica reglas), mientras que los mecanismos de comunicación deben ser ligeros, como el uso de mensajería HTTP con REST [8].
Gestión de datos descentralizada (Decentralized data management)	A diferencia de las arquitecturas monolíticas, cada microservicio debe gestionar su propia base de datos o repositorio de almacenamiento de forma independiente [8].
Diseño evolutivo (Evolutionary design)	Debido a su autonomía, los microservicios pueden evolucionar o ser reemplazados de forma independiente por nuevas implementaciones sin afectar a los consumidores del servicio [8].

3. Patrones de Arquitectura Distribuida: EDA, CQRS y API Gateway

Las arquitecturas de microservicios requieren patrones especializados para gestionar la comunicación, la consistencia de datos y el acceso de los clientes. A continuación, se analizan los cuatro patrones fundamentales indicados para esta unidad: EDA, CQRS, Saga y API Gateway.

3.1. Arquitectura Orientada a Eventos (EDA) con Kafka y RabbitMQ

Definición

La Arquitectura Orientada a Eventos (EDA) es un paradigma de diseño de software construido alrededor de la producción, detección, consumo y reacción a eventos discretos [9]. Se refiere a sistemas de microservicios que están débilmente acoplados e intercambian información o datos entre sí a través de eventos de publicación y escucha [10].

A diferencia de las arquitecturas basadas en API, que son sincrónicas, la EDA funciona de forma asíncrona, lo que permite reducir la latencia y los cuellos de botella de dependencia entre servicios [9], [10]. Las entidades principales en un sistema EDA son:

- **Productores de eventos:** Generan eventos basados en acciones internas o externas [9].
- **Consumidores de eventos:** Reciben y procesan estos eventos para ejecutar lógica de negocio [9].
- **Brokers (Intermediarios):** Manejan la entrega, enrutamiento, persistencia y confiabilidad de los mensajes entre productores y consumidores [9].

EDA con Apache Kafka y RabbitMQ

Ambos sistemas actúan como el “bus de eventos” o intermediarios, pero poseen características distintas según las necesidades del negocio:

Cuadro 4: Comparativa entre Apache Kafka y RabbitMQ.

Aspecto	Apache Kafka	RabbitMQ
Modelo	Funciona como una plataforma de transmisión de eventos distribuida diseñada para la ingesta y procesamiento de eventos en tiempo real de alto rendimiento [9].	Es un broker de mensajes basado en un modelo de colas tradicional que sigue protocolos como AMQP, MQTT y STOMP [9].
Persistencia	Utiliza un modelo de publicación-suscripción donde los eventos se escriben en registros inmutables llamados tópicos, los cuales se particionan y replican para asegurar la escalabilidad [9].	Los mensajes se enrutan a través de intercambios y se almacenan en colas hasta que son consumidos. Una vez entregado y confirmado, el mensaje suele eliminarse de la cola [9].
Casos de uso	Es ideal para tuberías de datos (pipelines), auditoría, análisis en tiempo real y escenarios que requieren reproducir mensajes (re-play), ya que permite a los consumidores leer el mismo mensaje varias veces [9].	Es preferido para la delegación de tareas y mensajería punto a punto, como la autorización de microtransacciones o el procesamiento de instrucciones de pago [9].
Escalabilidad	Soporta el manejo de millones de mensajes por segundo mediante el escalado horizontal [9].	Sobresale en la entrega de mensajes transaccionales con baja latencia y ofrece un control detallado sobre el enrutamiento y la confirmación de entrega [9].

Comparativa de rendimiento

En pruebas experimentales, se ha observado que la EDA implementada con estas herramientas ofrece un tiempo de respuesta más rápido (un 19.18 % de mejora) y una tasa de error menor (34.40 % menos) en comparación con las arquitecturas basadas en API, aunque el uso de recursos de CPU tiende a ser más alto [10]. Mientras que RabbitMQ es más estable que las comunicaciones REST tradicionales cuando hay un gran volumen de peticiones simultáneas, Kafka es superior para el procesamiento de flujos de datos a gran escala [9], [10].

3.2. Patrón CQRS, Patrón Saga y Patrón API Gateway

A continuación, se describe cada uno de estos patrones indicando su definición, cuándo usarlo, ventajas y desventajas.

1. Patrón CQRS (Command Query Responsibility Segregation)

- **Definición:** Es un patrón arquitectónico que divide el sistema en dos partes: el procesamiento de comandos (solicitudes de escritura/modificación del estado) y el

procesamiento de consultas (solicitudes de lectura) [11]. Separa la responsabilidad de la modificación de datos de su recuperación para que cada operación pueda ser optimizada de forma independiente [12].

- **Cuándo usarlo:** Se recomienda en sistemas a gran escala donde el volumen de transacciones y consultas puede abrumar los diseños monolíticos tradicionales [12]. Es ideal para escenarios de alta concurrencia de escritura y lectura, o cuando se requiere que las consultas tengan tiempos de respuesta rápidos sobre grandes volúmenes de datos [11].

Ventajas:

- Permite la optimización independiente de los modelos de lectura y escritura; por ejemplo, las lecturas pueden escalarse horizontalmente usando réplicas y caché sin afectar el rendimiento de las escrituras [12].
- Mejora el rendimiento, la escalabilidad y la seguridad [12].
- Facilita la integración con el “event sourcing”, mejorando la trazabilidad y auditabilidad [12].

Desventajas:

- Introduce una alta complejidad operativa y de mantenimiento debido a la necesidad de gestionar bases de datos y modelos separados [12].
- Requiere gestionar la “consistencia eventual” entre los modelos de comando y consulta, lo que puede generar anomalías temporales de datos y problemas de sincronización [12].

2. Patrón Saga

- **Definición:** Es una tecnología basada en el pensamiento orientado a eventos para implementar procesos de negocio de manera consistente [11]. Divide un proceso de negocio largo en múltiples transacciones independientes y pequeñas, donde cada paso es activado por un evento [9].
- **Cuándo usarlo:** Se utiliza para gestionar transacciones de negocio que abarcan múltiples microservicios y que requieren mantener la consistencia de los datos sin usar sistemas de transacciones distribuidas tradicionales [9]. Es útil cuando se deben realizar cambios en múltiples “raíces de agregación” [11].

Ventajas:

- Permite lograr una consistencia final (eventual consistency) en escenarios donde fallan las transacciones atómicas de un solo bloque [11].
- Mejora la resiliencia del sistema al disparar automáticamente acciones compensatorias (reversión) si un paso de la secuencia falla [9].

Desventajas:

- Depende críticamente de que la cola de mensajes sea altamente confiable; si se pierden mensajes, el proceso de la saga se rompe [11].
- Implica aceptar el retraso inherente de la consistencia final en lugar de tener datos actualizados instantáneamente en todas partes [11].

3. Patrón API Gateway

- **Definición:** Actúa como un punto de entrada único que ofrece los servicios necesarios para que las aplicaciones (clientes) interactúen con el backend [8]. En el caso de microservicios, puede presentarse como un “Backend for Frontend” (BFF), cuyo objetivo es desacoplar las aplicaciones frontend de la arquitectura backend [8].
- **Cuándo usarlo:** Se utiliza en arquitecturas de microservicios para centralizar el acceso de múltiples tipos de clientes, como aplicaciones web, móviles o de terceros [8].

Ventajas:

- Permite aislar el backend del frontend, facilitando cambios en los servicios internos sin afectar a los clientes [8].
- Ofrece APIs personalizadas y optimizadas para cada tipo de cliente, mejorando la experiencia del usuario [8].
- Promueve la reutilización de código al centralizar las llamadas de datos desde un servidor común [8].

Desventajas:

- El aumento en el número de microservicios consumidos desde el frontend puede incrementar la complejidad de la trazabilidad y la búsqueda de errores [8].

4. APIs RESTful: Diseño, Documentación y Seguridad con JWT

4.1. Principios REST de Fielding

El estilo arquitectónico REST (Representational State Transfer) establece seis restricciones fundamentales que toda API debe cumplir: interfaz uniforme, arquitectura cliente-servidor, ausencia de estado (statelessness), capacidad de caché, sistema en capas y código bajo demanda. Estas restricciones garantizan la escalabilidad, portabilidad e interoperabilidad entre sistemas distribuidos. Estudios empíricos recientes han demostrado que el cumplimiento de las guías de diseño REST incide directamente en la facilidad de uso de la API y en la reducción de errores de integración entre equipos [13].

4.2. Diseño de Recursos y URIs

El diseño de recursos es el núcleo de cualquier API REST. Un recurso es cualquier entidad de negocio identificable (usuario, producto, pedido) que puede ser manipulada a través de su URI. Las URIs deben construirse usando sustantivos en plural (p.ej. /usuarios, /productos/{id}/categorias), reflejar jerarquías lógicas y evitar verbos en la ruta. Investigaciones aplicadas en integración de datos mediante REST confirman que una correcta estructuración de las URIs y el uso del formato JSON reducen tiempos de respuesta y el consumo de CPU del servidor [14].

4.3. Verbos HTTP: Semántica e Idempotencia

Los métodos HTTP definen la acción que se ejecuta sobre un recurso. Cada método posee propiedades específicas de seguridad e idempotencia que deben respetarse para construir APIs predecibles y robustas. La siguiente tabla presenta sus características:

Cuadro 5: *Semántica, idempotencia y seguridad de los verbos HTTP* [15].

Verbo	Semántica	¿Idempotente?	¿Seguro?
GET	Recupera un recurso. No produce efectos secundarios en el servidor.	Sí	Sí
POST	Crea un nuevo recurso. Cada invocación puede producir un recurso distinto.	No	No
PUT	Reemplaza completamente un recurso existente.	Sí	No
PATCH	Actualiza parcialmente un recurso. Generalmente idempotente según implementación.	Generalmente sí	No
DELETE	Elimina un recurso. Eliminar un recurso ya eliminado devuelve el mismo estado final.	Sí	No

Un análisis de arquitecturas API en aplicaciones de producción confirma que el uso incorrecto de los verbos HTTP (p.ej., emplear GET para operaciones con efectos secundarios) es una de las causas más frecuentes de comportamientos inesperados en sistemas RESTful [15].

4.4. Documentación con OpenAPI 3.0

OpenAPI Specification (OAS) 3.0 es el estándar de facto para describir APIs RESTful de forma legible tanto por humanos como por máquinas. Un documento OpenAPI (YAML o JSON) define endpoints, operaciones, parámetros de entrada, esquemas de respuesta, mecanismos de autenticación (p.ej., Bearer JWT) y ejemplos de error. Herramientas como Swagger UI, Redoc y Postman consumen estas especificaciones para generar portales de documentación interactivos y colecciones de prueba automáticamente. La extracción automática de endpoints REST desde el código fuente de microservicios y su conversión a archivos OpenAPI válidos ha sido validada mediante agentes LLM, reduciendo la brecha entre implementación y documentación [16].

4.5. Autenticación JWT: Estructura y Flujo Completo

JSON Web Token (JWT), definido en el RFC 7519, es el mecanismo de autenticación sin estado más adoptado en APIs REST modernas. Un JWT está compuesto por tres segmentos codificados en Base64URL separados por puntos, como se muestra en la siguiente tabla:

Cuadro 6: *Estructura de un JSON Web Token (JWT).*

Segmento	Contenido	Ejemplo / Campos comunes
Header	Especifica el algoritmo de firma y el tipo de token.	<code>{".alg": "HS256", "typ": "JWT"}</code>
Payload	Contiene los claims: información del sujeto y metadatos temporales.	<code>sub, roles, iat, exp, nbf</code>
Signature	Garantiza la integridad del token, calculada sobre header.payload con clave secreta o clave privada RSA.	<code>HMACSHA256(base64(header)+". "+base64(payload), secret)</code>

El flujo completo opera así: (1) el cliente envía credenciales al endpoint de autenticación; (2) el servidor valida y emite un access token (corta duración) y un refresh token (larga duración); (3) el cliente incluye el access token en el encabezado **Authorization: Bearer <token>** en cada solicitud; (4) el servidor verifica la firma y los claims sin consultar la base de datos; (5) al expirar el access token, el refresh token permite obtener uno nuevo. Estudios comparativos demuestran que el uso de JWT combinado con cifrado AES-256 en el payload mitiga ataques de robo de tokens y exposición de información sensible [17], mientras que técnicas de randomización criptográfica dinámica y detección de anomalías mediante redes LSTM incrementan aún más la resiliencia del esquema de autenticación [18].

4.6. Comparación REST vs. gRPC vs. GraphQL

La elección del paradigma de comunicación entre microservicios tiene un impacto relevante en el rendimiento, la flexibilidad y la mantenibilidad del sistema. La siguiente tabla resume las diferencias entre las tres alternativas más utilizadas:

Cuadro 7: Comparativa entre REST, gRPC y GraphQL.

Criterio	REST	gRPC	GraphQL
Protocolo	HTTP/1.1 o HTTP/2	HTTP/2 exclusivo	HTTP/1.1 o HTTP/2
Serialización	JSON / XML (texto)	Protocol Buffers (binario)	JSON
Rendimiento	Mayor throughput en baja concurrencia [19].	276 KB/s vs. 134 KB/s de REST; mejor latencia estable [19].	Comparable a REST; overhead de resolver.
Flexibilidad de consulta	Endpoints fijos; puede generar over/under-fetching.	Contratos fijos definidos en .proto.	El cliente solicita exactamente los campos necesarios [19].
Ideal para	APIs públicas de propósito general.	Microservicios internos de alta frecuencia.	Aplicaciones con necesidades de datos altamente variables.
Soporte de streaming	Limitado (Server-Sent Events).	Streaming bidireccional nativo.	Suscripciones (limitado).

5. Contenerización con Docker y Orquestación con Kubernetes

5.1. Diferencias entre Máquinas Virtuales y Contenedores

Las máquinas virtuales (VM) virtualizan el hardware completo e incluyen un sistema operativo guest íntegro, lo que implica un consumo elevado de recursos y tiempos de arranque de varios minutos. Los contenedores, en cambio, comparten el kernel del sistema operativo anfitrión y encapsulan únicamente el código de la aplicación y sus dependencias, lo que los hace extremadamente ligeros (arranque en milisegundos), portables y eficientes. El aislamiento se logra mediante namespaces y cgroups de Linux. La adopción de tecnologías de contenerización con Docker y orquestación con Kubernetes representa un avance fundamental para la migración de aplicaciones cloud hacia entornos perimetrales (edge computing) en la industria [20].

5.2. Arquitectura Docker: Daemon, Imagen, Contenedor y Registro

La arquitectura de Docker se compone de cuatro elementos fundamentales que interactúan para gestionar el ciclo de vida completo de los contenedores:

Cuadro 8: Componentes fundamentales de la arquitectura Docker.

Componente	Descripción
Docker Daemon (dockerd)	Proceso en segundo plano que gestiona imágenes, contenedores, redes y volúmenes mediante la API REST de Docker [21].
Imagen	Plantilla de solo lectura que define el sistema de archivos del contenedor; construida en capas superpuestas (layers) mediante un Dockerfile [21].
Contenedor	Instancia en ejecución de una imagen; efímero por naturaleza. El estado persistente se gestiona con volúmenes [21].
Registro (Registry)	Repositorio de imágenes. Docker Hub es el registro público; existen privados: Harbor, AWS ECR, Azure ACR [21].

5.3. Dockerfile: Instrucciones y Buenas Prácticas

Un Dockerfile es un archivo de texto con instrucciones secuenciales para construir una imagen Docker. Las instrucciones fundamentales son:

- **FROM:** Define la imagen base (p.ej., `node:20-alpine`).
- **WORKDIR:** Establece el directorio de trabajo dentro del contenedor.
- **COPY / ADD:** Transfiere archivos del host al sistema de archivos de la imagen.
- **RUN:** Ejecuta comandos durante la construcción (instalación de dependencias).
- **EXPOSE:** Documenta el puerto de escucha de la aplicación.
- **CMD / ENTRYPOINT:** Define el comando ejecutado al iniciar el contenedor.

Las buenas prácticas incluyen: minimizar capas agrupando comandos RUN, usar imágenes base ligeras (Alpine Linux), implementar builds multi-etapa (multi-stage builds) para separar entornos de compilación y producción, y nunca incluir credenciales en el Dockerfile. La propuesta de plataformas educativas basadas en microservicios orquestados en contenedores Docker demuestra la viabilidad de estas prácticas en proyectos de producción reales [22].

5.4. Docker Compose: Aplicaciones Multi-contenedor

Docker Compose permite definir y ejecutar aplicaciones multi-contenedor mediante un archivo declarativo (`docker-compose.yml`) que especifica servicios, redes, volúmenes y variables de entorno. Con un único comando (`docker compose up`) se despliega toda la pila coordinada. Es la herramienta ideal para entornos de desarrollo y pruebas locales donde conviven bases de datos, cachés, colas de mensajes y aplicaciones backend/frontend. Investigaciones sobre orquestación en entornos Minikube + Kubernetes + Docker Compose evidencian que la combinación de estas herramientas permite despliegues fluidos y escalado dinámico de arquitecturas de microservicios [23].

5.5. Introducción a Kubernetes: Pods, Deployments, Services e Ingress

Kubernetes (K8s) es la plataforma de orquestación de contenedores open source más adoptada a escala mundial. Su arquitectura distingue un plano de control (API Server, Scheduler, Controller Manager, etcd) y nodos de trabajo. Los objetos clave son:

Cuadro 9: Objetos fundamentales de Kubernetes y sus funciones.

Objeto K8s	Función
Pod	Unidad mínima de despliegue; encapsula uno o más contenedores que comparten red y almacenamiento [24].
Deployment	Declara el estado deseado de los Pods; gestiona réplicas, rolling updates y rollbacks automáticos [24].
Service	Abstracción que expone Pods a la red; provee balanceo de carga y descubrimiento de servicios estable [24].
Ingress	Gestiona acceso HTTP/S externo al clúster con reglas de enrutamiento y terminación TLS [24].
ConfigMap / Secret	Injectan configuraciones y credenciales de forma desacoplada de las imágenes de contenedor [24].

La implementación de middleware semántico sobre Docker y Kubernetes proporciona nuevos niveles de eficiencia, escalabilidad y portabilidad en el diseño de sistemas distribuidos [24].

5.6. Patrones de Despliegue: Blue-Green y Canary

Los patrones de despliegue avanzados permiten lanzar nuevas versiones de software en producción sin interrupciones del servicio:

Blue-Green Deployment: Se mantienen dos entornos de producción idénticos (azul = versión actual; verde = nueva versión). La nueva versión se despliega y valida en el entorno verde; al confirmar su correcto funcionamiento, el enrutador de tráfico conmuta de forma instantánea. En caso de fallo, el rollback consiste en redirigir el tráfico de vuelta al entorno azul, logrando tiempos de recuperación casi nulos.

Canary Deployment: La nueva versión se despliega inicialmente a un subconjunto reducido de usuarios. El sistema monitoriza métricas de rendimiento y tasa de errores; si los indicadores son satisfactorios, el porcentaje de tráfico dirigido a la nueva versión se incrementa progresivamente hasta el 100 %. Este patrón detecta errores en producción real con impacto mínimo. Herramientas como Argo Rollouts, Flagger e Istio automatizan estos procesos en Kubernetes. Estudios comparativos sobre estrategias de despliegue continuo en DevOps confirman que los despliegues canary y blue-green optimizan la disponibilidad del sistema y reducen el riesgo de regresiones en entornos cloud-native [25].

6. Arquitectura del Sistema

6.1. Diagrama C4 – Nivel 2: Contenedores

La Figura 1 muestra la arquitectura a nivel de contenedores del sistema TiendaTech. El cliente realiza todas sus peticiones a través del API Gateway (Nginx), que enruta el tráfico a los microservicios correspondientes según el prefijo de la ruta.

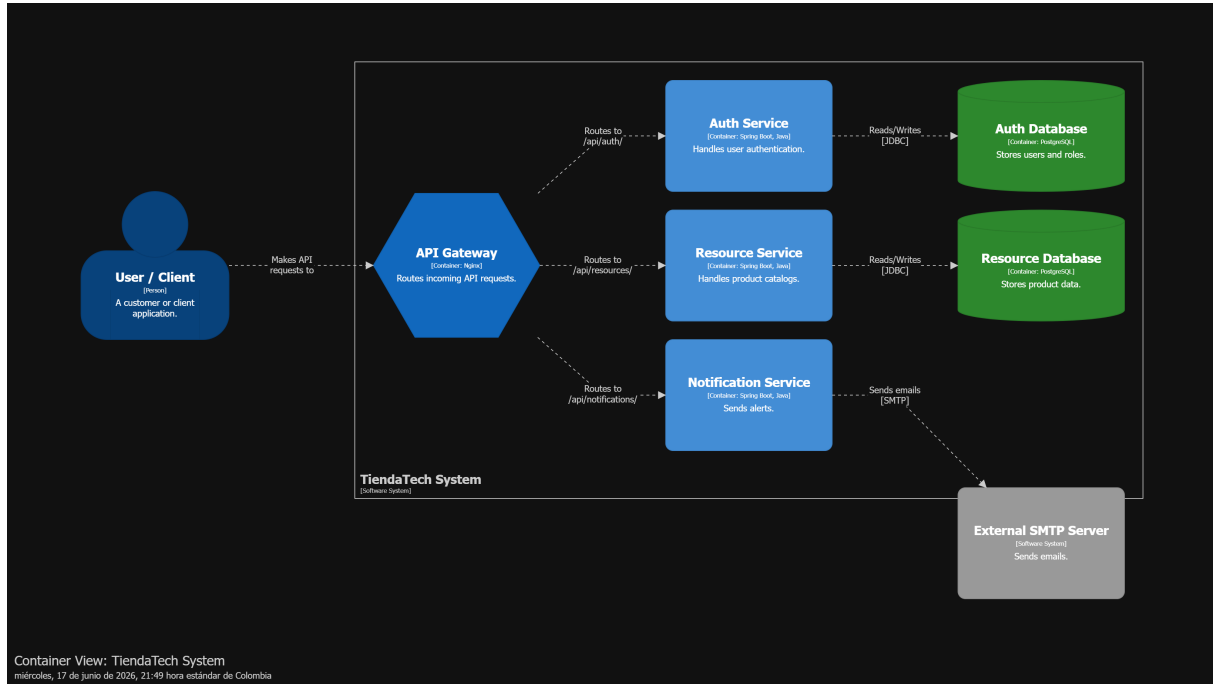


Figura 1: Arquitectura TiendaTech – Nivel 2: Contenedores (C4)

6.2. Contratos OpenAPI 3.0

A continuación se presentan los contratos de cada microservicio en formato YAML.

6.2.1. Auth Service

Listing 1: Contrato OpenAPI 3.0 – Auth Service (fragmento)

```
openapi: 3.0.0
info:
  title: Auth Service API
  version: 1.0.0
paths:
  /register:
    post:
      summary: Registrar nuevo usuario
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
```



```

        required: [username, password]
        properties:
            username: { type: string, minLength: 3 }
            password: { type: string, minLength: 6 }
    responses:
        '201':
            description: Usuario registrado exitosamente
/login:
    post:
        summary: Iniciar sesion y obtener JWT
        responses:
            '200':
                description: Token JWT con campo expiresIn
            '401':
                description: Credenciales invalidas
/validate:
    get:
        summary: Validar token JWT via header Authorization
        responses:
            '200':
                description: Token valido, retorna username y valid=true
            '401':
                description: Token invalido o expirado

```

6.2.2. Resource Service

Listing 2: Contrato OpenAPI 3.0 – Resource Service (fragmento)

```

openapi: 3.0.0
info:
    title: Resource Service API
    version: 1.0.0
security:
    - bearerAuth: []
paths:
    /:
        get:
            summary: Listar todos los productos
            responses:
                '200': { description: Lista de productos }
                '401': { description: Token no proporcionado }
    /create:
        post:
            summary: Crear producto (dispara notificacion)
            responses:
                '201': { description: Producto creado }
    /{id}:
        get:
            summary: Obtener producto por ID
        put:
            summary: Actualizar producto (dispara notificacion)
        delete:

```

```

        summary: Eliminar producto (dispara notificacion)
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

```

6.2.3. Notification Service

Listing 3: Contrato OpenAPI 3.0 – Notification Service (fragmento)

```

openapi: 3.0.0
info:
  title: Notification Service API
  version: 1.0.0
paths:
  /notifications:
    post:
      summary: Registrar y persistir evento de notificacion
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                resourceId: { type: integer }
                action:
                  type: string
                  enum: [PRODUCT_CREATED, PRODUCT_UPDATED,
                        PRODUCT_DELETED, USER_REGISTERED]
                message: { type: string }
                recipientEmail: { type: string }
      responses:
        '201': { description: Notificacion persistida en DB }
    get:
      summary: Consultar historial de notificaciones
      responses:
        '200': { description: Lista de NotificationLog desde PostgreSQL }

```

7. Implementación

7.1. Auth Service

El auth-service implementado en Spring Boot 3 con Java 21 expone tres endpoints: `POST /register` para crear nuevos usuarios con contraseña cifrada en BCrypt, `POST /login` que valida credenciales y emite un JWT firmado con HS512 con expiración de una hora, y `GET /validate` que verifica la firma y vigencia del token.

Listing 4: AuthController – endpoint de registro

```

@PostMapping("/register")
public ResponseEntity<ApiResponse<Map<String, Object>>> register(
    @RequestBody RegisterRequest request) {
    try {
        AppUser user = authService.register(request);
        Map<String, Object> data = Map.of(
            "username", user.getUsername(),
            "roles", user.getRoles()
        );
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(new ApiResponse<>(201, data,
                "Usuario registrado exitosamente"));
    } catch (RuntimeException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(new ApiResponse<>(409, null, ex.getMessage()));
    }
}

```

Listing 5: JwtTokenProvider – generación de token

```

public String generateToken(Authentication authentication) {
    String username = authentication.getName();
    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + jwtExpirationMs);
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(now)
        .setExpiration(expiryDate)
        .signWith(signingKey, SignatureAlgorithm.HS512)
        .compact();
}

```

7.2. Resource Service

El resource-service implementa CRUD completo para el recurso Product (nombre, descripción, precio, stock, categoría). Cada endpoint verifica el JWT en el encabezado `Authorization: Bearer <token>`. Tras operaciones de escritura, invoca de forma asíncrona al notification-service mediante NotificationClient.

Listing 6: ProductService – creación con notificación

```

public Product createProduct(Product product) {
    Product saved = productRepository.save(product);
    notificationClient.notify(saved.getId(), "PRODUCT_CREATED",
        "Producto creado: " + saved.getName());
    return saved;
}

```

Listing 7: NotificationClient – llamada HTTP al notification-service

```

public void notify(Long resourceId, String action, String message)
{

```

```

try {
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    Map<String, Object> body = Map.of(
        "resourceId", resourceId,
        "action", action,
        "message", message,
        "recipientEmail", "admin@tiendatech.com"
    );
    HttpEntity<Map<String, Object>> request =
        new HttpEntity<>(body, headers);
    restTemplate.postForEntity(
        notificationServiceUrl + "/notifications",
        request, String.class);
} catch (Exception ex) {
    logger.warn("No se pudo enviar notificación: {}",
        ex.getMessage());
}
}

```

7.3. Notification Service

El notification-service persiste cada evento en su base de datos PostgreSQL propia (notification-db) mediante la entidad JPA NotificationLog y simula el envío de correo electrónico registrando el mensaje en los logs del contenedor.

Listing 8: NotificationService – persistencia en DB

```

public NotificationLog sendNotification(
    NotificationEventRequest request) {
    NotificationLog log = new NotificationLog(
        request.getResourceId(),
        request.getAction(),
        request.getMessage(),
        request.getRecipientEmail()
    );
    simulateSmtpEmailSend(log);
    return repository.save(log); // Persiste en notification-db
}

```

8. Configuración del API Gateway (Nginx)

Nginx escucha en el puerto 8080 y aplica tres políticas transversales: enrutamiento, rate limiting (máximo 100 peticiones por minuto por IP con burst de 10) y logging estructurado en formato JSON.

Listing 9: nginx.conf – configuración del API Gateway

```

http {
    log_format microservices_log
        '{"time": "$time_iso8601", "remote_addr": "$remote_addr", '

```

```

        "request_method": "$request_method",
        "request_uri": "$request_uri", "status": $status}';

limit_req_zone $binary_remote_addr zone=perip:10m rate=100r/m;

upstream auth_service      { server auth-service:8001; }
upstream resource_service  { server resource-service:8002; }
upstream notification_service { server notification-service
                               :8003; }

server {
    listen 8080;

    location /api/auth/ {
        limit_req zone=perip burst=10 nodelay;
        proxy_pass http://auth_service;
        proxy_set_header X-Real-IP $remote_addr;
    }
    location /api/resources/ {
        limit_req zone=perip burst=10 nodelay;
        proxy_pass http://resource_service;
    }
    location /api/notifications/ {
        limit_req zone=perip burst=10 nodelay;
        proxy_pass http://notification_service;
    }
}
}

```

9. Contenerización con Docker Compose

9.1. Dockerfile Multi-Stage

Todos los microservicios utilizan el mismo patrón Dockerfile multi-stage con dos etapas: *builder* (Maven 3.9 + JDK 21 Alpine) y *runtime* (JRE 21 Alpine). Esto reduce la imagen final de aproximadamente 700 MB a menos de 200 MB.

Listing 10: Dockerfile multi-stage (auth-service)

```

# Etapa 1: Builder
FROM maven:3.9.9-eclipse-temurin-21-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

# Etapa 2: Runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/auth-service-0.0.1-SNAPSHOT.jar app
.jar

```

```
EXPOSE 8001
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

9.2. Docker Compose

El archivo `docker-compose.yml` define siete servicios (tres microservicios, tres bases de datos y Nginx), todos conectados a la red interna `tiendatech-network` de tipo *bridge*.

Listing 11: `docker-compose.yml` (fragmento `notification-db`)

```
notification-db:
  image: postgres:15-alpine
  container_name: notification-db
  environment:
    POSTGRES_USER: notif_user
    POSTGRES_PASSWORD: SecureNotifPass789!@#
    POSTGRES_DB: notificationdb
  volumes:
    - notification_data:/var/lib/postgresql/data
  networks:
    - tiendatech-network

notification-service:
  build:
    context: ./notification-service
  environment:
    - NOTIF_DATABASE_URL=jdbc:postgresql://notification-db:5432/notificationdb
    - NOTIFICATION_SERVICE_URL=http://notification-service:8003
  depends_on:
    - notification-db
  networks:
    - tiendatech-network
```

Para levantar el sistema completo se ejecuta:

Listing 12: Comando de despliegue

```
docker-compose up --build
```

10. Resultados de Pruebas – Flujos End-to-End con Postman

Las pruebas se ejecutaron directamente contra los contenedores corriendo con `docker-compose up -build`. Todos los tests automatizados de la colección Postman resultaron en estado **PASS**. Las capturas de pantalla correspondientes a cada flujo se encuentran en el [Anexo A](#).

10.1. Flujo 1 – Registro y Login JWT

Cuadro 10: Flujo 1: Registro y obtención de JWT

Endpoint	Método	Esperado	Obtenido	Latencia
http://localhost:8001/login	POST	200	200	134 ms
http://localhost:8001/validate	GET	200	200	56 ms

Tests PASS verificados (Flujo 1):

- **PASS** – Status 200 - Login exitoso
- **PASS** – Respuesta tiene token JWT
- **PASS** – Token tiene expiresIn
- **PASS** – Status 200 - Token válido
- **PASS** – valid = true en respuesta
- **PASS** – Tiene username en data

10.2. Flujo 2 – CRUD de Productos con JWT

Cuadro 11: Flujo 2: CRUD completo con validación JWT

Endpoint	Método	Esperado	Obtenido	Latencia
http://localhost:8002/ (sin token)	GET	401	401	10 ms
http://localhost:8002/ (con token)	GET	200	200	533 ms

Tests PASS verificados (Flujo 2):

- **PASS** – Status 401 - Sin token rechazado
- **PASS** – Mensaje de error presente
- **PASS** – Status 200 - Con token aceptado
- **PASS** – Data es array

10.3. Flujo 3 – Crear Recurso y Verificar Notificación

Cuadro 12: Flujo 3: Creación de producto y verificación de notificación

Endpoint	Método	Esperado	Obtenido	Latencia
http://localhost:8002/create	POST	201	201	52 ms
http://localhost:8003/notifications	GET	200	200	29 ms

Tests PASS verificados (Flujo 3):

- **PASS** – Status 201 - Producto creado
- **PASS** – Guardar ID del producto
- **PASS** – Status 200 - Historial obtenido
- **PASS** – Historial tiene al menos una notificación
- **PASS** – Última notificación es PRODUCT_CREATED

10.4. Resumen General de Pruebas

Cuadro 13: Resumen consolidado de resultados – todos los flujos

Endpoint	Método	Esperado	Obtenido	Resultado	Latencia
/login	POST	200	200	PASS	134 ms
/validate	GET	200	200	PASS	56 ms
/ (sin token)	GET	401	401	PASS	10 ms
/ (con token)	GET	200	200	PASS	533 ms
/create	POST	201	201	PASS	52 ms
/notifications	POST	201	201	PASS	29 ms
POST					
/notifications	GET	200	200	PASS	29 ms
GET					
Total tests				11/11 PASS	–

11. Análisis – Preguntas Obligatorias

11.1. ¿Por qué cada microservicio debe tener su propia base de datos?

El patrón *Database per Service* garantiza el desacoplamiento completo entre microservicios a nivel de datos. Si dos servicios comparten la misma base de datos, un cambio de esquema en uno puede romper al otro sin posibilidad de despliegue independiente. Además, cada servicio puede elegir la tecnología de persistencia más adecuada para su dominio (PostgreSQL, MongoDB, Redis) sin afectar a los demás. En TiendaTech, los tres microservicios utilizan instancias PostgreSQL separadas (auth-db, resource-db, notification-db).

11.2. ¿Qué ocurre si resource-service cae? Circuit Breaker

Si el **resource-service** cae, las peticiones entrantes acumularán timeouts, lo que puede cascadear hacia el API Gateway y degradar toda la plataforma. El patrón **Circuit Breaker** mitiga este problema: mantiene tres estados (Closed, Open, Half-Open) y, tras un umbral de fallos consecutivos, abre el circuito rechazando peticiones inmediatamente con una respuesta de fallback (por ejemplo, lista de productos vacía en caché). Para implementarlo en Spring Boot se usaría **Resilience4j**:

Listing 13: Circuit Breaker con Resilience4j (ejemplo)

```
@CircuitBreaker(name = "resourceService",
                fallbackMethod = "productsFallback")
public List<Product> getProducts() {
    return restTemplate.getForObject(
        resourceServiceUrl + "/", List.class);
}

public List<Product> productsFallback(Exception ex) {
    logger.warn("CircuitBreaker activo: {}", ex.getMessage());
}
```



```
        return Collections.emptyList();
    }
```

11.3. ¿Introduce el API Gateway un cuello de botella?

El API Gateway puede convertirse en un cuello de botella si es un único nodo sin estado horizontal. En TiendaTech, Nginx maneja hasta varias decenas de miles de conexiones concurrentes gracias a su arquitectura de event loop no bloqueante, por lo que el riesgo en un escenario académico es bajo. Para minimizarlo en producción se recomendaría: (1) desplegar múltiples réplicas del Gateway con un load balancer externo; (2) activar caché de respuestas en Nginx para endpoints GET de alta frecuencia; y (3) utilizar HTTP/2 para multiplexar peticiones sobre una sola conexión TCP.

11.4. ¿Cómo escalar resource-service a 3 réplicas con Docker Compose?

Para escalar en Docker Compose se elimina el `container_name` fijo del servicio y se añade la directiva `deploy`:

Listing 14: Escalar resource-service a 3 réplicas

```
resource-service:
  build:
    context: ./resource-service
  deploy:
    replicas: 3
  networks:
    - tiendatech-network
```

En Nginx se actualiza el bloque `upstream` para usar el nombre de servicio de Docker (que resuelve automáticamente entre réplicas con round-robin DNS):

Listing 15: Nginx upstream con balanceo de carga

```
upstream resource_service {
    server resource-service:8002; # Docker resuelve las 3 replicas
}
```

12. Discusión

La implementación realizada permitió comprobar de manera práctica varios de los conceptos estudiados sobre arquitecturas distribuidas y microservicios. La separación de funcionalidades en servicios independientes facilitó la organización de la aplicación y permitió asignar responsabilidades específicas a cada componente, reduciendo la dependencia directa entre ellos.

Uno de los aspectos más relevantes observados fue la utilidad del patrón *Database per Service*, ya que cada microservicio gestionó sus propios datos sin interferir con los demás. De igual manera, la incorporación de JWT permitió proteger los recursos del

sistema mediante un mecanismo de autenticación sencillo y ampliamente utilizado en aplicaciones modernas.

Por otra parte, el uso de Nginx como API Gateway contribuyó a centralizar las solicitudes realizadas por los clientes, simplificando el acceso a los distintos servicios y permitiendo aplicar políticas comunes de control y monitoreo. Asimismo, Docker y Docker Compose facilitaron considerablemente el despliegue del sistema, reduciendo problemas relacionados con configuraciones del entorno y dependencias de software.

13. Conclusiones

1. Se implementó satisfactoriamente una arquitectura basada en microservicios compuesta por servicios independientes de autenticación, gestión de recursos y notificaciones, logrando una adecuada separación de responsabilidades dentro del sistema.
2. La utilización de JWT y de un API Gateway permitió centralizar el acceso a los servicios y mejorar la seguridad de las operaciones realizadas por los usuarios.
3. La contenerización mediante Docker y Docker Compose facilitó el despliegue y la administración de la aplicación, permitiendo ejecutar todos los componentes de manera integrada y consistente.
4. Los resultados obtenidos durante las pruebas funcionales evidenciaron el correcto funcionamiento de la solución implementada y demostraron la viabilidad del uso de microservicios para el desarrollo de aplicaciones distribuidas escalables y mantenibles.

Bibliografía

- [1] M. Kaloudis, «Evolving Software Architectures from Monolithic Systems to Resilient Microservices: Best Practices, Challenges and Future Trends,» *International Journal of Advanced Computer Science and Applications*, vol. 15, n.º 9, págs. 1-10, 2024. DOI: [10.14569/IJACSA.2024.0150901](https://doi.org/10.14569/IJACSA.2024.0150901).
- [2] N. Kratzke, «A Brief History of Cloud Application Architectures,» *Applied Sciences*, vol. 8, n.º 8, págs. 3-26, 2018. DOI: [10.3390/app8081368](https://doi.org/10.3390/app8081368).
- [3] V. Benavente, L. Yantas, I. Moscol, C. Rodriguez, R. Inquilla e Y. Pomachagua, «Comparative Analysis of Microservices and Monolithic Architecture,» en *2022 14th International Conference on Computational Intelligence and Communication Networks (CICN)*, Saudi Arabia, 2022. DOI: [10.1109/CICN56167.2022.10008275](https://doi.org/10.1109/CICN56167.2022.10008275).
- [4] G. Blinowski, A. Ojdowska y A. Przybyłek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: [10.1109/ACCESS.2022.3152803](https://doi.org/10.1109/ACCESS.2022.3152803).
- [5] S. Weerasinghe e I. Perera, «An exploratory evaluation of replacing ESB with microservices in service oriented architecture,» en *International Research Conference on Smart Computing and Systems Engineering 2021*, 2021. DOI: [10.1109/SCSE53661.2021.9568289](https://doi.org/10.1109/SCSE53661.2021.9568289).

- [6] T. Cerny, M. J. Donahoo y J. Pechanec, «Disambiguation and Comparison of SOA, Microservices and Self-Contained Systems,» en *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*, Krakow, 2017. DOI: [10.1145/3129676.3129682](https://doi.org/10.1145/3129676.3129682).
- [7] M. Khanbabaei y M. Asadi, «Principles of service-oriented architecture and web services application in order to implement service-oriented architecture in software engineering,» *Australian Journal of Basic and Applied Sciences*, vol. 5, n.º 11, págs. 2046-2051, 2011, ISSN: 1991-8178.
- [8] V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. DOI: [10.1109/ACCESS.2023.3305687](https://doi.org/10.1109/ACCESS.2023.3305687).
- [9] O. T. Odofin, S. Owode, E. Ogbuefi, J. C. Ogeawuchi, O. S. Adanigb y T. P. Gbenle, «Integrating Event-Driven Architecture in Fintech Operations Using Apache Kafka and RabbitMQ Systems,» *International Journal of Multidisciplinary Research and Growth Evaluation*, vol. 3, n.º 4, págs. 635-643, 2022. DOI: [10.54660/.IJMRGE.2022.3.4.635-643](https://doi.org/10.54660/.IJMRGE.2022.3.4.635-643).
- [10] A. Rahmatulloh, F. Nugraha, R. Gunawan e I. Darmawan, «Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems,» en *2022 International Conference Advancement in Data Science, E-learning and Information Systems (ICADEIS)*, 2022. DOI: [10.1109/ICADEIS56544.2022.10037390](https://doi.org/10.1109/ICADEIS56544.2022.10037390).
- [11] ZhangLong, «Improvement and Implementation of a High Performance CQRS Architecture,» en *2017 International Conference on Robots & Intelligent System (ICRIS)*, 2017. DOI: [10.1109/ICRIS.2017.49](https://doi.org/10.1109/ICRIS.2017.49).
- [12] S. Jayaraman y R. Mishra, «Implementing Command Query Responsibility Segregation (CQRS) in Large-Scale Systems,» *International Journal of Research in Modern Engineering and Emerging Technology (IJRMEET)*, vol. 12, n.º 12, págs. 49-73, 2024, ISSN: 2320-6586.
- [13] M. Coblenz, W. Guo, K. Voozhian y J. S. Foster, «A Qualitative Study of REST API Design and Specification Practices,» en *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, Washington, DC, USA, 2023. DOI: [10.1109/VL-HCC57772.2023.00025](https://doi.org/10.1109/VL-HCC57772.2023.00025).
- [14] M. K. Yusof, M. Man y A. Ismail, «Design and Implement of REST API for Data Integration,» en *2022 International Conference on Engineering and Emerging Technologies (ICEET)*, Kuala Lumpur, Malaysia, 2023. DOI: [10.1109/ICEET56468.2022.10007414](https://doi.org/10.1109/ICEET56468.2022.10007414).
- [15] K. Kumar, A. K. Jain, R. G. Tiwari, N. Jain, V. Gautam y N. K. Trivedi, «Analysis of API Architecture: A Detailed Report,» en *2023 IEEE 12th International Conference on Communication Systems and Network Technologies (CSNT)*, Bhopal, India, 2023. DOI: [10.1109/CSNT57126.2023.10134658](https://doi.org/10.1109/CSNT57126.2023.10134658).
- [16] O. Chaplia y H. Klym, «Extracting REST API Endpoints from Microservices Using LLM Agents,» en *2024 14th International Conference on Dependable Systems, Services and Technologies (DESSERT)*, Athens, Greece, 2025. DOI: [10.1109/DESSERT65323.2024.11122242](https://doi.org/10.1109/DESSERT65323.2024.11122242).

- [17] K. Widatama y P. W. Setyaningsih, «Development of A Secure API Authentication Mechanism with JWT and Data Encryption,» en *2025 2nd International Conference on Information System and Information Technology (ICISIT)*, Yogyakarta, Indonesia, 2025. DOI: [10.1109/ICISIT66233.2025.11402947](https://doi.org/10.1109/ICISIT66233.2025.11402947).
- [18] S. Bhuiya, G. M. Shakil, G. M. M. Bashir y M. Samsuzzaman, «Ejwt: Enhancing JWT Security through Dynamic Cryptographic & LSTM Networks,» en *2026 5th International Conference on Electrical, Computer & Telecommunication Engineering (ICECTE)*, Rajshahi, Bangladesh, 2026. DOI: [10.1109/ICECTE69292.2026.11429432](https://doi.org/10.1109/ICECTE69292.2026.11429432).
- [19] K. Nieman y S. Sajal, «A Comparative Analysis on Load Balancing and gRPC Microservices in Kubernetes,» en *2023 Intermountain Engineering, Technology and Computing (IETC)*, Provo, UT, USA, 2023. DOI: [10.1109/IETC57902.2023.10152023](https://doi.org/10.1109/IETC57902.2023.10152023).
- [20] M. S. A. Muthanna y A. Tselykh, «Development of Docker and Kubernetes Orchestration Platforms for Industrial Internet of Things Service Migration,» en *2022 International Conference on Modern Network Technologies (MoNeTec)*, Moscow, Russian Federation, 2022. DOI: [10.1109/MoNeTec55448.2022.9960769](https://doi.org/10.1109/MoNeTec55448.2022.9960769).
- [21] K. P. Sah, N. Jain, P. Jha, J. Hawari y B. Beena, «Advancing of Microservices Architecture with Dockers,» en *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, Kamand, India, 2024. DOI: [10.1109/ICCCNT61001.2024.10724035](https://doi.org/10.1109/ICCCNT61001.2024.10724035).
- [22] A. Naik, J. Choudhari, V. Pawar y S. Shitole, «Building an EdTech Platform Using Microservices and Docker,» en *2021 IEEE Pune Section International Conference (PuneCon)*, Pune, India, 2021. DOI: [10.1109/PuneCon52575.2021.9686535](https://doi.org/10.1109/PuneCon52575.2021.9686535).
- [23] F. Eyvazov, T. E. Ali, F. I. Ali y A. D. Zoltan, «Beyond Containers: Orchestrating Microservices with Minikube, Kubernetes, Docker, and Compose for Seamless Deployment and Scalability,» en *2024 11th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)*, Noida, India, 2024. DOI: [10.1109/ICRITO61523.2024.10522382](https://doi.org/10.1109/ICRITO61523.2024.10522382).
- [24] Y. Madani y A. Akanbi, «A Semantic Middleware using Docker and Kubernetes Orchestration Tools,» en *2024 IST-Africa Conference (IST-Africa)*, Dublin, Ireland, 2024. DOI: [10.23919/IST-Africa63983.2024.10569625](https://doi.org/10.23919/IST-Africa63983.2024.10569625).
- [25] I. Gorton, Y. Tong y S. Yao, «Distributed Systems - Concepts Every Software Architect Should Know,» en *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, Hyderabad, India, 2024. DOI: [10.1109/ICSA-C63560.2024.00069](https://doi.org/10.1109/ICSA-C63560.2024.00069).

Anexo A – Capturas de Pruebas Postman

Las siguientes imágenes corresponden a las capturas obtenidas durante la ejecución de los flujos de prueba *end-to-end* descritos en la sección 10 de este documento.

A.1 Flujo 1 – Registro y Login JWT

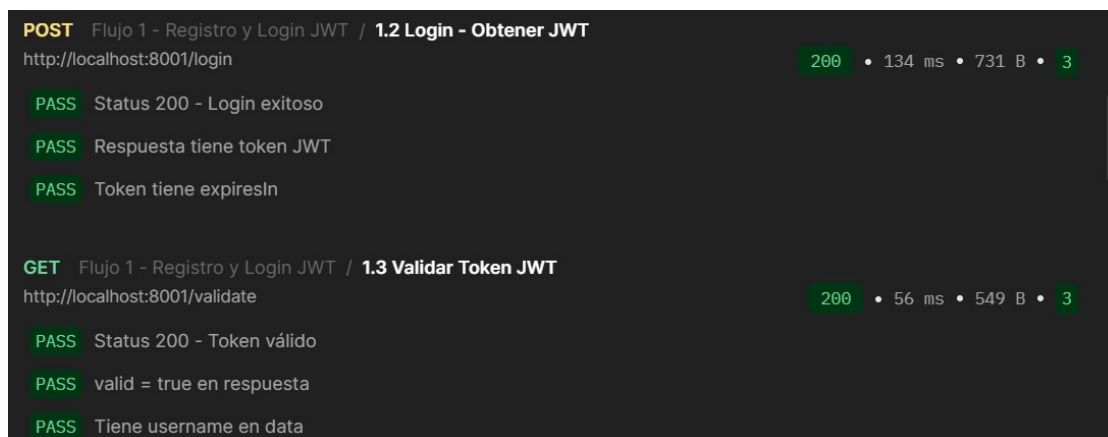


Figura 2: Captura Postman: Flujo 1 – Registro y obtención de JWT.

A.2 Flujo 2 – CRUD de Productos con JWT

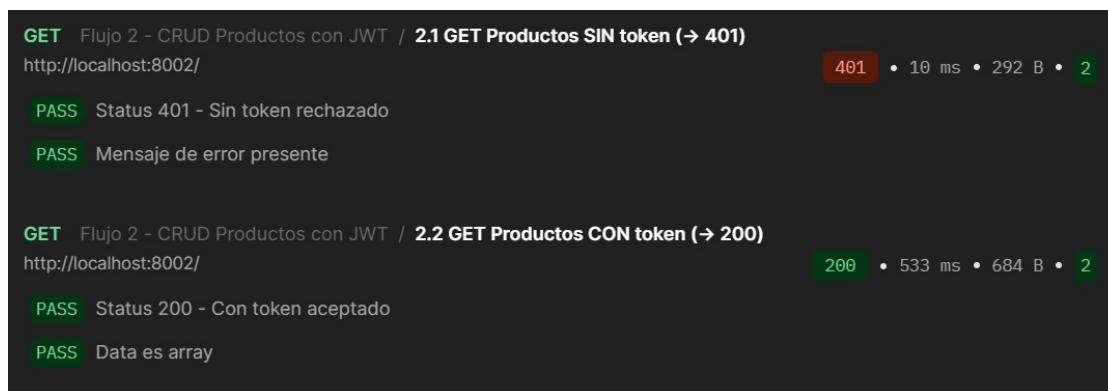


Figura 3: Captura Postman: Flujo 2 – CRUD de productos (parte 1).

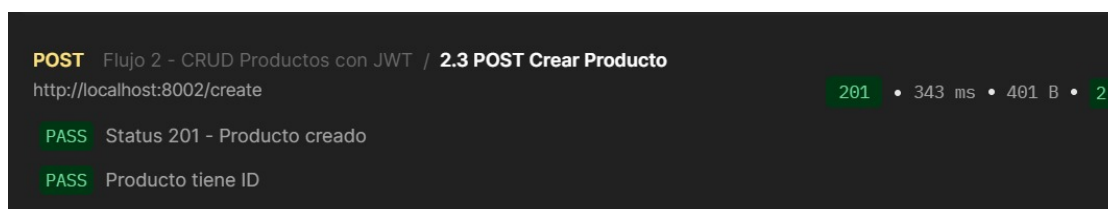


Figura 4: Captura Postman: Flujo 2 – CRUD de productos (parte 2).

A.3 Flujo 3 – Crear Recurso y Verificar Notificación

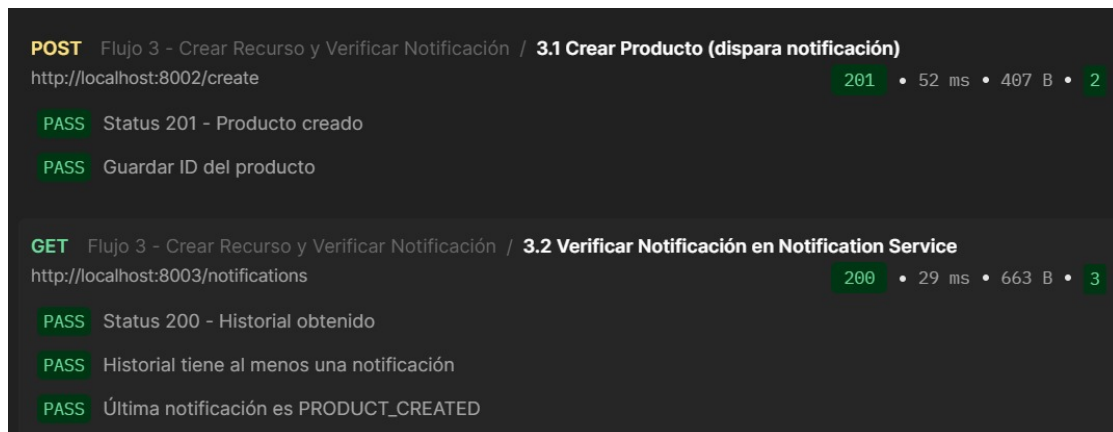


Figura 5: Captura Postman: Flujo 3 – Creación de recurso y verificación de notificación.